Welcome — AI workshop

Michal Lison · 2026-05-20

- Hands-on — laptops out, we build things
- Three breaks; longer one mid-day
- Interrupt anytime — questions, tangents, "wait, what?"

Repo: https://github.com/mlison/ai-workshop-public

Today

- Module 3 — Team context
- Module 4 — Giving the agent your team's tools (MCP)
- Module 5 — Discipline
- Module 6 — AI in CI/CD

Three breaks. End at 16:00.

What's worth learning today

The AI tooling landscape changes monthly. Which means half of what we cover today will be **flavour-of-the-month**: specific CLI flags, model names, plugin APIs.

The other half is **durable** — won't be obsoleted by the next model release:

- Team-level vocabulary and conventions
- Spec-first discipline
- Verification loops
- Cost-aware automation patterns

We'll flag which is which as we go. Lean into the durable.

Module 3

Team-level Context Engineering

50 minutes

Making AI consistent across your team

Same vocabulary. Same standards. Same conventions.

The ceiling on team productivity isn't model intelligence —
it's how clearly you can collectively express how you work.

*This is where you start being able to **trust each other's AI output** — because you've all agreed on what good looks like.*

Three layers of team-level context

1. **Persona / communication** — how the agent talks. Direct or cushioned. Push back, never flatter.
2. **Process / workflow** — how work gets done. Spec-first. PR conventions.
3. **Project conventions / standards** — file naming. Library defaults. What to never use.

Persona makes the difference

Same prompt: *"Thinking of using* `localStorage` *to keep an operator's edit-in-progress in the dashboard form. Good plan?"*

Without a persona (model default):

> *"Yes, common pattern! You can persist form state in localStorage. Handle JSON serialization, set a key, clear on submit. Want me to show you the code?"*

With DispatchKit's AGENTS.md persona:

> *"Pushback. Two problems: (1) sensitive operator data leaks across tabs and shared machines — for a dispatch dashboard that's not OK; (2) localStorage is synchronous and blocks the main thread on every keystroke. Alternatives: sessionStorage, IndexedDB, or a server-side draft endpoint. Which matters more — privacy or implementation simplicity?"*

The maintenance rule

AGENTS.md — how we work

doc/ — what the system does

Update each when its concern changes.
Without the rule, both rot.

Team AI configuration

AGENTS.md — always on

.skills/ — task-scoped, invoked

.prompts/ — reusable canvases

All three versioned. All three reviewed in PRs.

Before you read — meet DispatchKit

Fictional product the sample is written for.

- **Internal operations platform** for a logistics fleet — vehicles, drivers, routes, dispatch, alerts.
- **Multi-package TypeScript monorepo** — operator dashboard, thin API, driver app, shared protocol.
- **Team** owns the dashboard and API; *not* the upstream Dispatch Core source-of-truth.

You'll see this product again in Modules 4-6. Same team, same repo.

Critique: would you ship this?

`modules/module-3/sample/AGENTS.md`

1. What would feel off using this with your team tomorrow?
2. What's missing for a team-level (vs individual) artifact?
3. What rots fastest?
4. What hidden assumption wouldn't transfer to your team?

Open Claude. Have it read the file with you. Push back. Ask "what would *you* cut?"

Where good prompts come from

You don't have to write every prompt from scratch.

- **Anthropic Prompt Library** — first-party, tested patterns.
- **Prompting Guide** — techniques and prompt-engineering reference.
- **Your team's `.prompts/`** — the one that compounds.

Browse first. Adapt second. Don't reinvent.

Coming up: Module 4

- Mob-build an MCP tool.
- Write a skill solo.

`.prompts/` you take home.

10-minute break.